Conntrazy Work Log

Scope reference

- Source references:
  - `Conntrazy Developer Handbook.md`
  - `Conntrazy Client Proposal.md`
- Focus: **Full MVP End-to-End Build**

Phase 1: Foundation (Week 1)

1. **Project conversion baseline**
   - Bootstrapped **Next.js 16.2.4** App Router.
   - Configured Tailwind + shadcn/ui.
2. **Architecture setup**
   - Added boundaries (`src/features`, `src/lib`, `src/app`).
   - Setup Redux toolkit.
3. **Authentication foundation**
   - Configured NextAuth (Google + Credentials).
   - Roles model implementation.
   - Super admin via env vars.
4. **Database foundation**
   - Added complete Prisma schema to Neon PostgreSQL.
   - Pushed basic tables (`User`, `VendorProfile`).

Phase 2: Vendor Workspace & Setup (Week 2)

1. **Stripe Connect**
   - Completed `/vendor/stripe` page.
   - Built `connect`, `return`, and `refresh` API routes for standard onboarding.
   - Status updates automatically in vendor profile.
2. **Contract & Checklist Management**
   - Built `/vendor/contracts` UI & APIs to manage `ContractTemplate`.
   - Built `/vendor/checklists` UI & APIs to manage required documents (`ChecklistTemplate`, `ChecklistItem`).
3. **Transaction Creation Engine**
   - Implemented `/vendor/actions` generating unique secure tokens for `TransactionLink`.
   - Added QR code generation (`qrcode.react`) on transaction creation.
   - Automatically maps selected contracts and checklist requirements to the generated `Transaction`.

Phase 3: Client Journey & Payment (Week 3 & 4)

1. **Secure Link Access & Profile**
    - Developed tokenized middleware via `getTransactionByToken` and `validateClientStep` to strictly enforce the client flow sequence.
    - Built manual profile collection at `/t/[token]/profile` with idempotency updates (upsert) to prevent duplicate profiles per transaction.
2. **Identity Verification (KYC)**
    - Implemented real Stripe Identity integration at `/api/client/[token]/kyc` creating verification sessions.
    - Handled KYC return callback to update `KycVerification` status.
3. **Document & Photo Uploads**
    - Built `/t/[token]/documents` with Cloudinary direct upload integration.
    - Maps successful uploads to `DocumentAsset` in database.
4. **Contract Auto-Population & Signature**
    - Implemented regex-based variable replacement for `{{clientName}}`, `{{paymentAmount}}`, etc.
    - Built signature checkbox confirmation at `/t/[token]/contract`.
5. **Stripe Checkout & Deposits**
    - Built `/t/[token]/payment` and `/api/client/[token]/checkout`.
    - Differentiates between standard payment (automatic capture) and deposit holding (manual capture).
    - Added Vendor Dashboard controls to **Release** or **Capture** authorized deposits manually via Stripe intents api (`/api/vendor/transactions/[id]/deposit`).
6. **Timeline, Webhooks, & Emails**
    - Built `/vendor/transactions/[id]` for vendors to monitor client progress live.
    - Set up `/api/webhooks/stripe` to handle async session completion.
    - Wired transactional emails via **Resend**, sending completion receipts to clients and deposit alerts to vendors natively upon success.

Validation Status

- ☑ `npm run build` passes cleanly.
- ☑ Full database schema synchronized.
- ☑ All Phase 1, Phase 2, and Phase 3 MVP goals are completed according to the Proposal and Developer Handbook.